

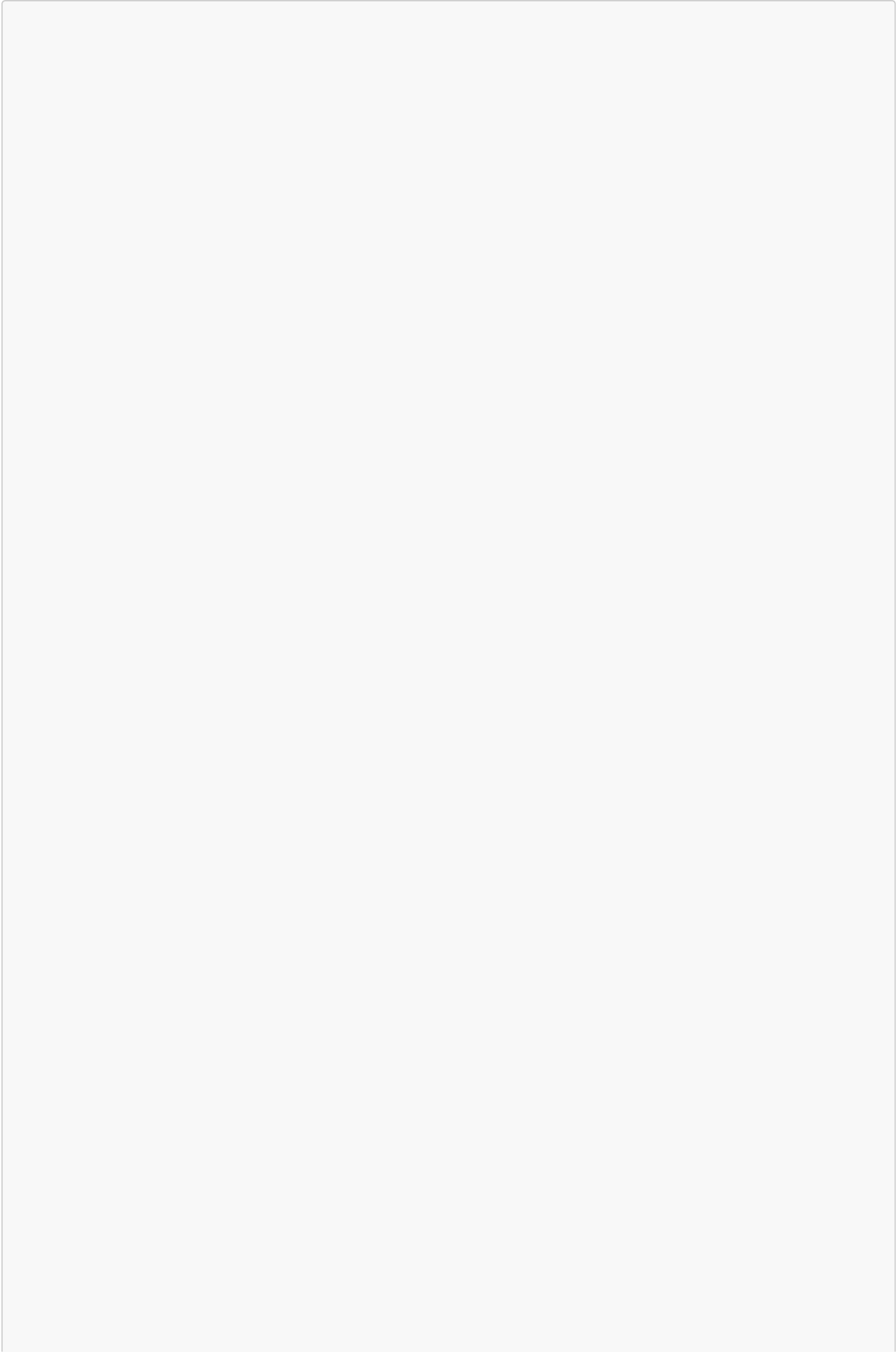
DEV_DOC.md 図解ガイド

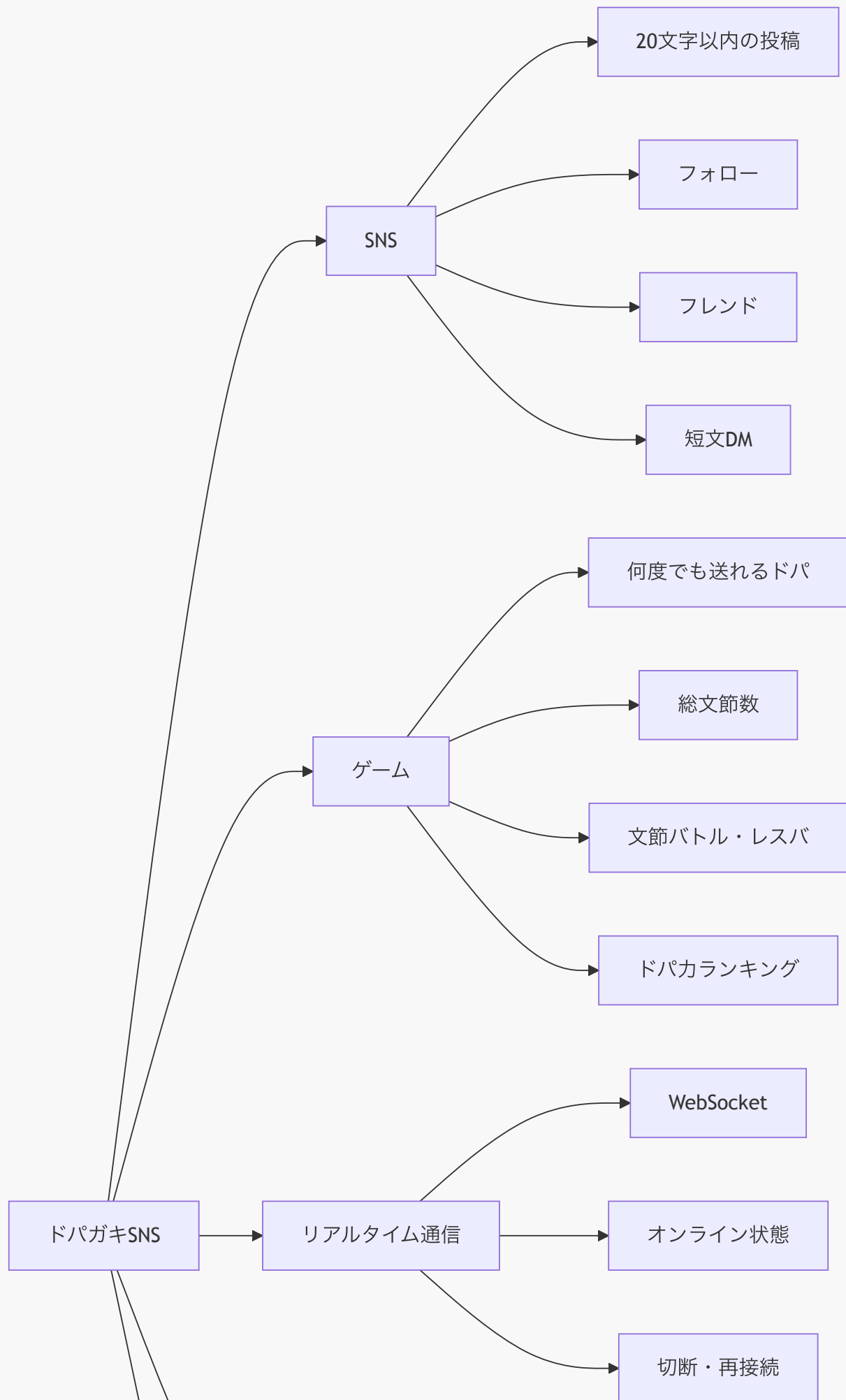
原文: ../DEV_DOC.md

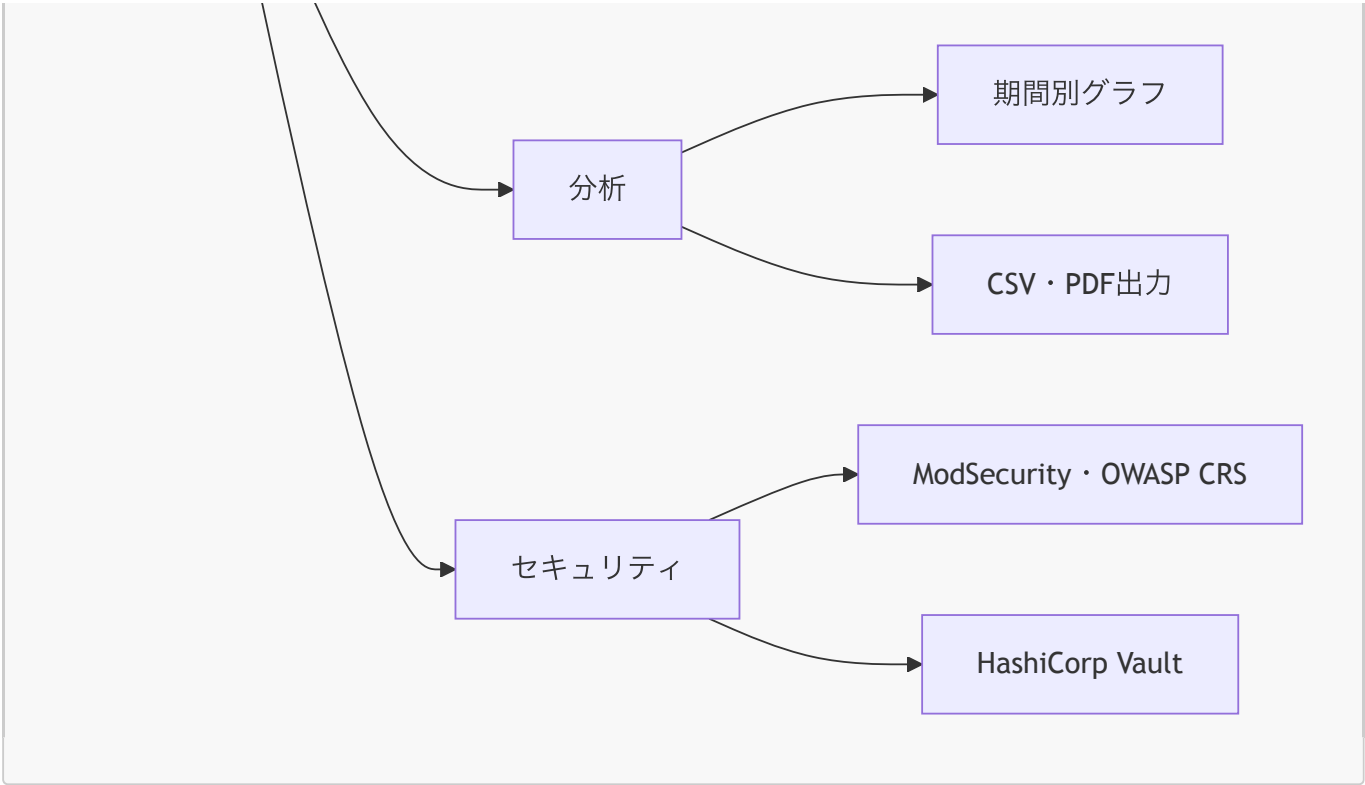
この文書は、原文の設計意図と機能同士の関係を素早く理解するための補助資料です。仕様の最終判断では原文を優先してください。

1. 何を作るのか

「ドパガキSNS」は、短文SNS、ユーザー交流、リアルタイム対戦、分析、セキュリティを一つにしたWebアプリケーションです。





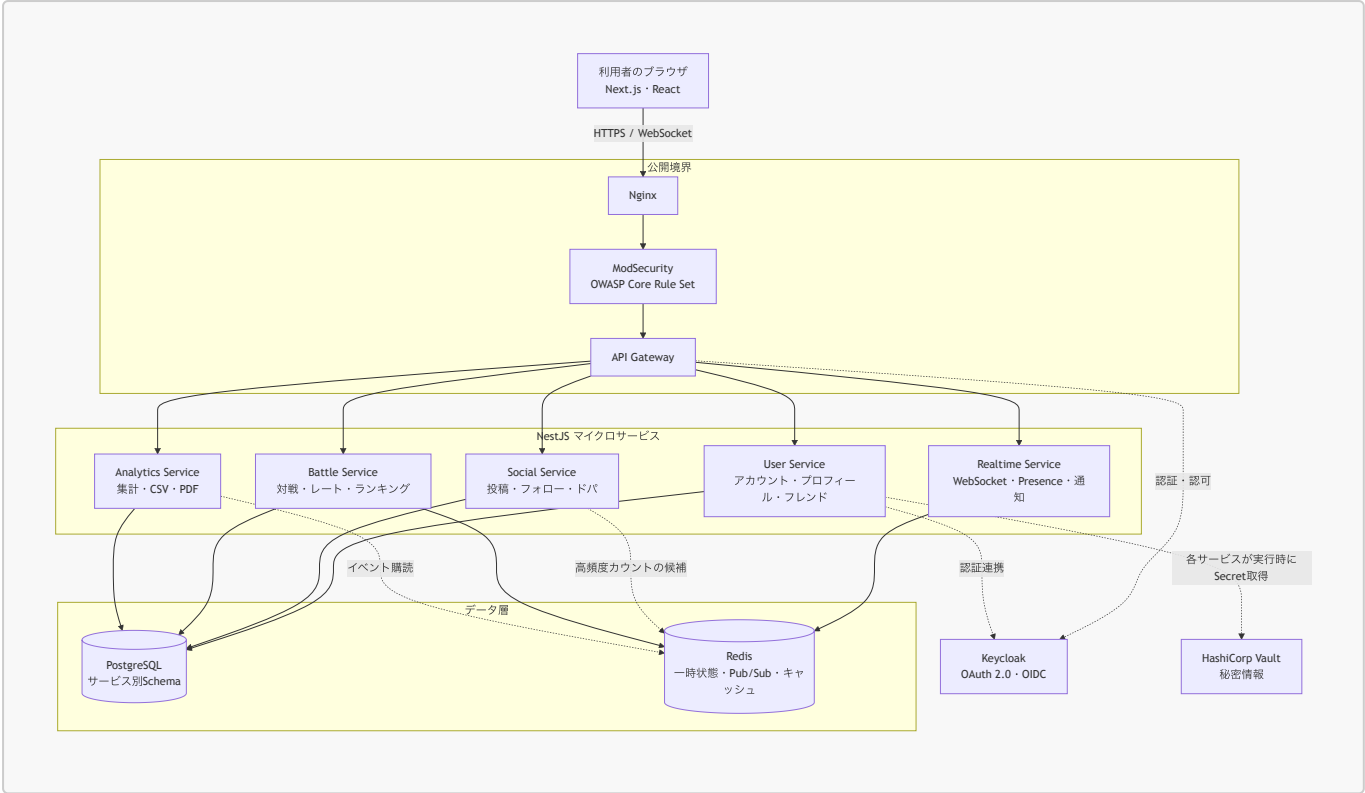


このサービス独自のルール

操作・概念 一般的なSNSとの違い	
投稿	本文は最大20文字。成功するとゲーム上のフォロワーが5,000増える
文節	投稿の文節数を数え、ユーザーの「総文節数」に加算する
ドバ	いいねの代替。同じユーザーが同じ投稿へ何度でも送れる
文節バトル	総文節数が多いユーザーが勝つ
ドパレート	バトルに勝つと上がり、負けると下がる
ドパカ	総文節数、ドパレート、戦績、連勝数などから算出する強さ
フォロワー	実際のフォロー関係とは別のゲーム用数値。投稿やバトルで変動する

2. システム全体像

外部通信はNginxとWAFを通し、API Gatewayが各サービスへ振り分けます。PostgreSQLは単一インスタンスを共有しますが、サービスごとにSchemaを分離します。

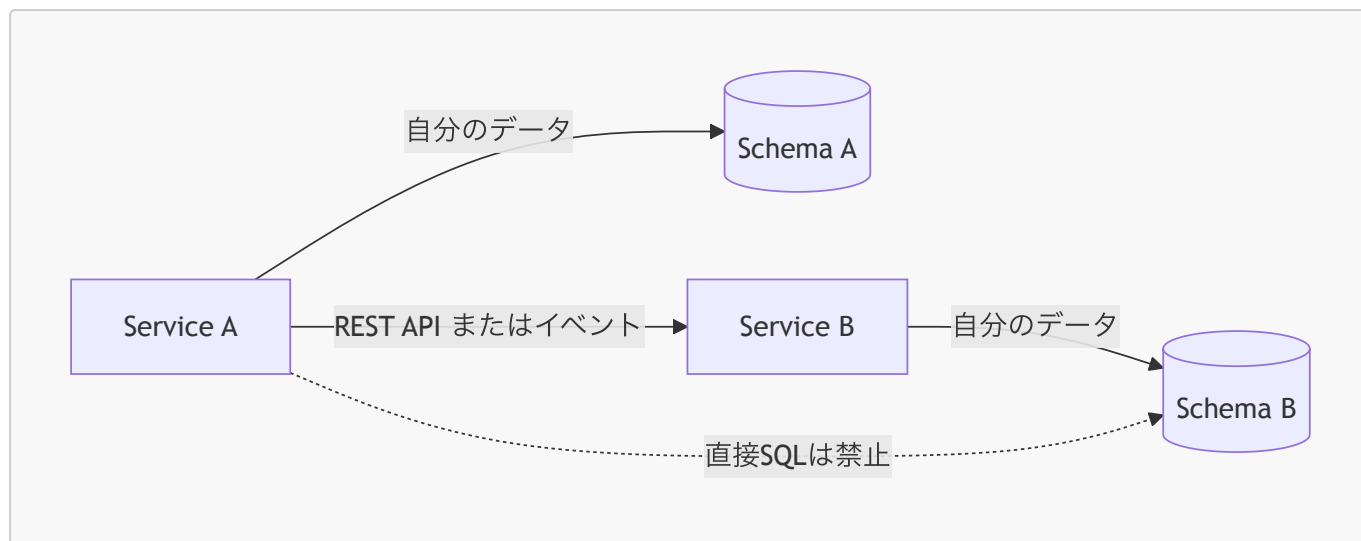


サービスの責務と所有データ

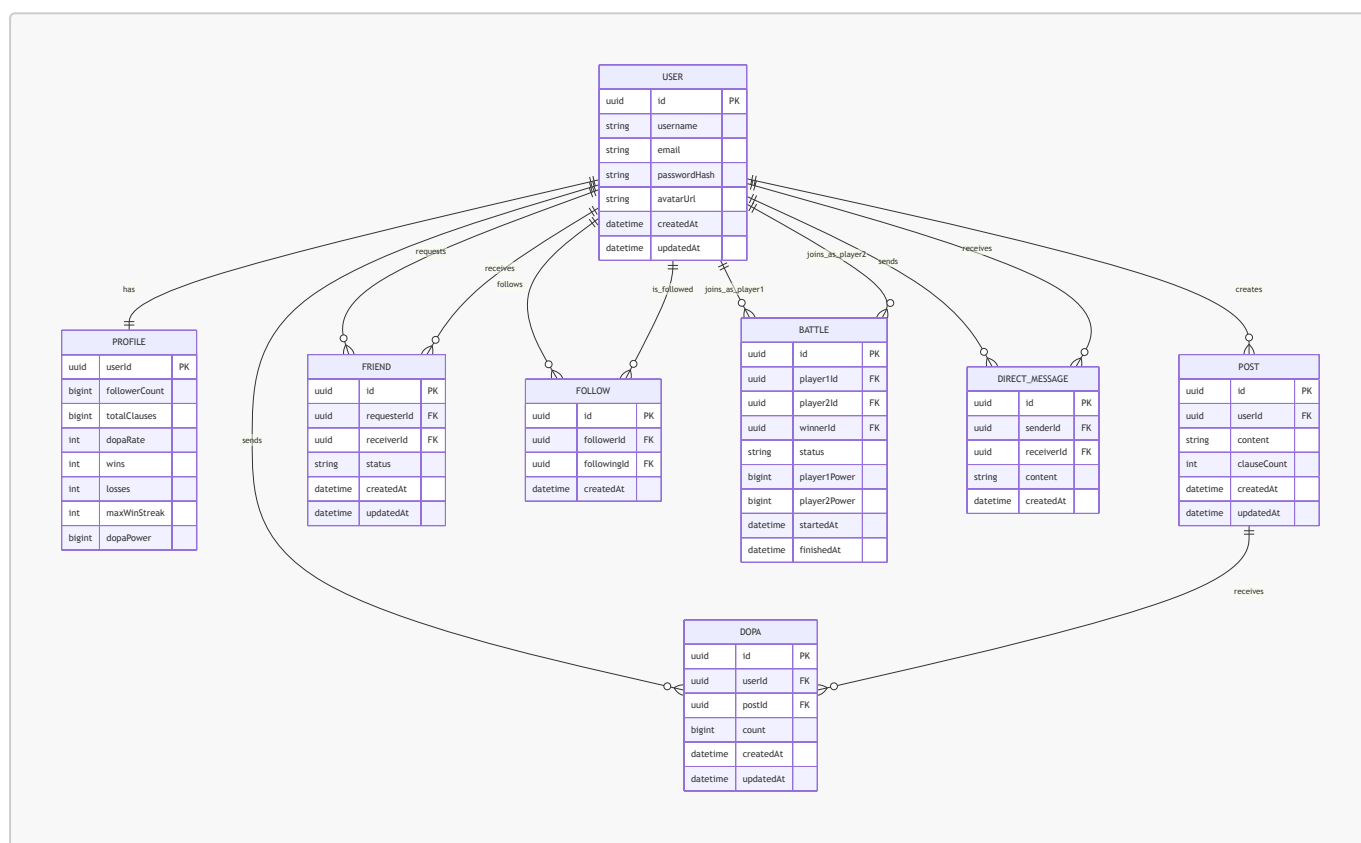
サービス	主な責務	主に扱うデータ・状態
User Service	登録、ログイン、プロフィール、アバター、フレンド、オンライン情報	User、Profile、Friend
Social Service	投稿、タイムライン、フォロー、総文節数、ドパ	Post、Follow、Dopa、投稿関連プロフィール値
Battle Service	バトル、マッチング、勝敗、レート、ドパカ、ランキング	Battle、対戦中状態、戦績
Realtime Service	WebSocket、通知、状態同期、切断・再接続	接続情報、Presence、配信用イベント
Analytics Service	各種集計、期間フィルター、グラフ、CSV・PDF	集計値、時系列データ

サービスは自分のSchemaだけを直接操作し、他サービスのデータはAPIまたはイベント経由で利用します。

原文の論理モデルでは、一つの **Profile** にSocial領域の **followerCount**・**totalClauses** とBattle領域の **dopaRate**・戦績・**dopaPower** が入っています。このまま複数サービスが同じテーブルを更新するとSchema所有の原則と衝突します。実装前に、統計をサービス別テーブルへ分割するか、**Profile** の更新担当を一つにしてAPI・イベント経由で更新するかを決める必要があります。



3. データモデル

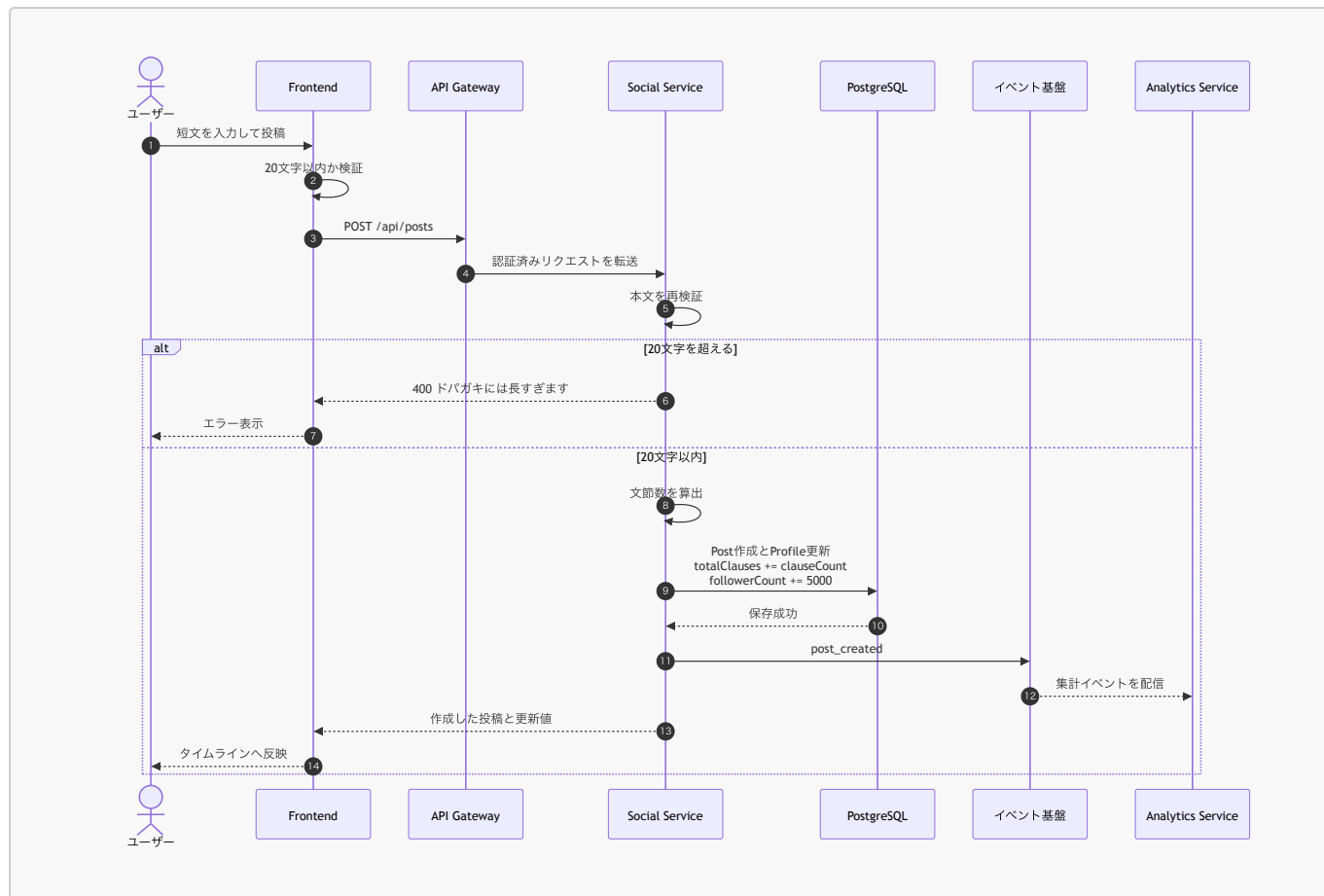


重要な区別は次のとおりです。

- **Follow** は「誰が誰をフォローしているか」というSNS上の関係です。
- **Profile.followerCount** は投稿で +5,000 され、バトルでも増減するゲーム上の数値です。
- **Dopa** は一人一回のLikeではありません。複数回送信でき、大きな値になる前提です。
- 永続データはPostgreSQLへ保存し、Redisを永続DBの代わりにはしません。

4. 投稿すると何が起きるか

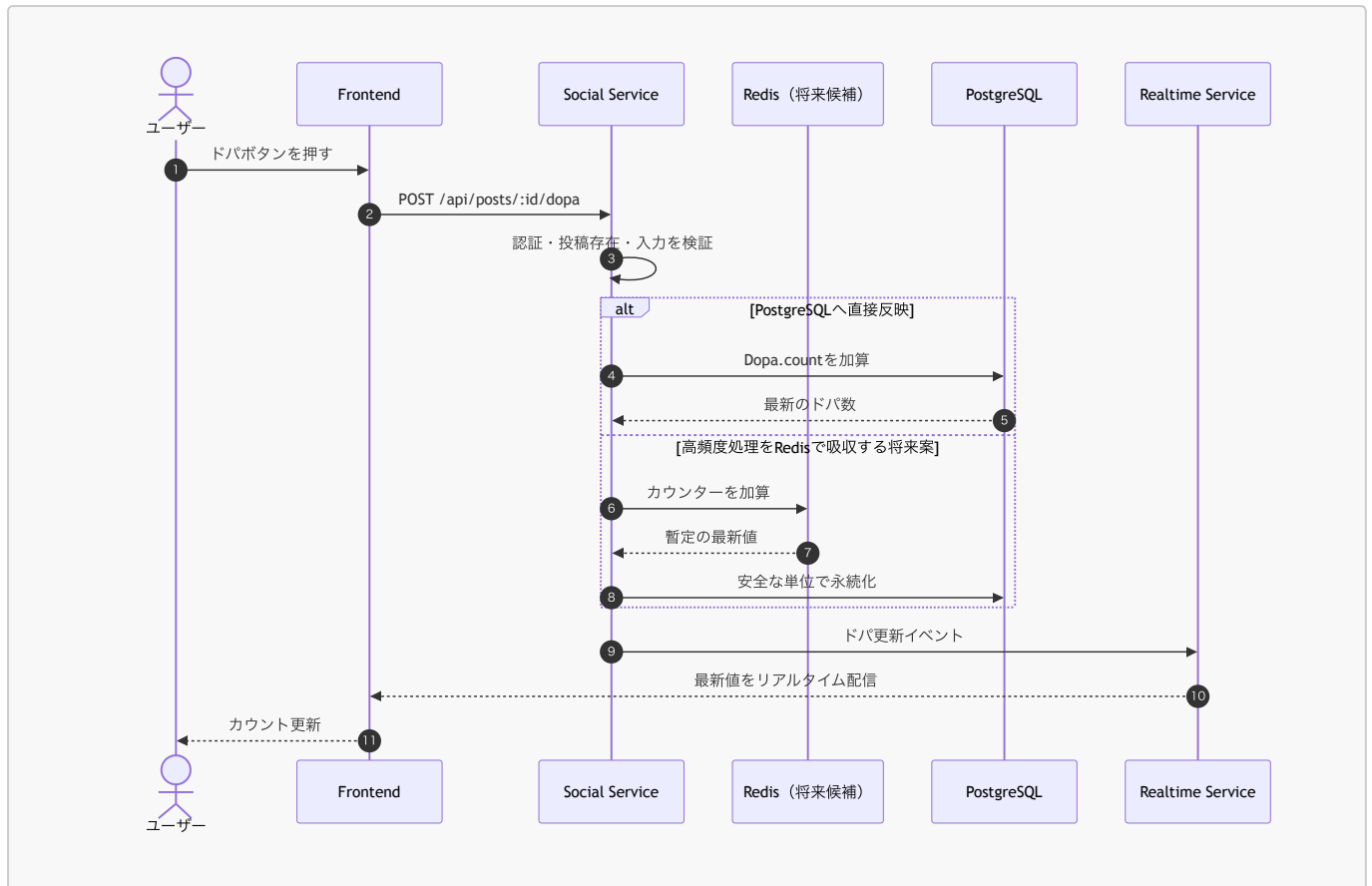
投稿の作成、文節数の集計、ゲーム用フォロワーの加算は、一つの操作として整合性を保つ必要があります。



フロントエンドの文字数チェックは操作性のため、バックエンドのチェックは不正防止のために必要です。クライアントが送る **clauseCount** や加算後の値を信用せず、サーバー側で計算します。

図は投稿時に必要な業務更新をまとめて示しています。サービス別Schemaを厳密に分ける場合、Social Serviceが他サービス所有の **Profile** を直接更新してはいけません。Profileの所有者へAPI・イベントで依頼するか、Social Service所有の統計テーブルとして分離します。

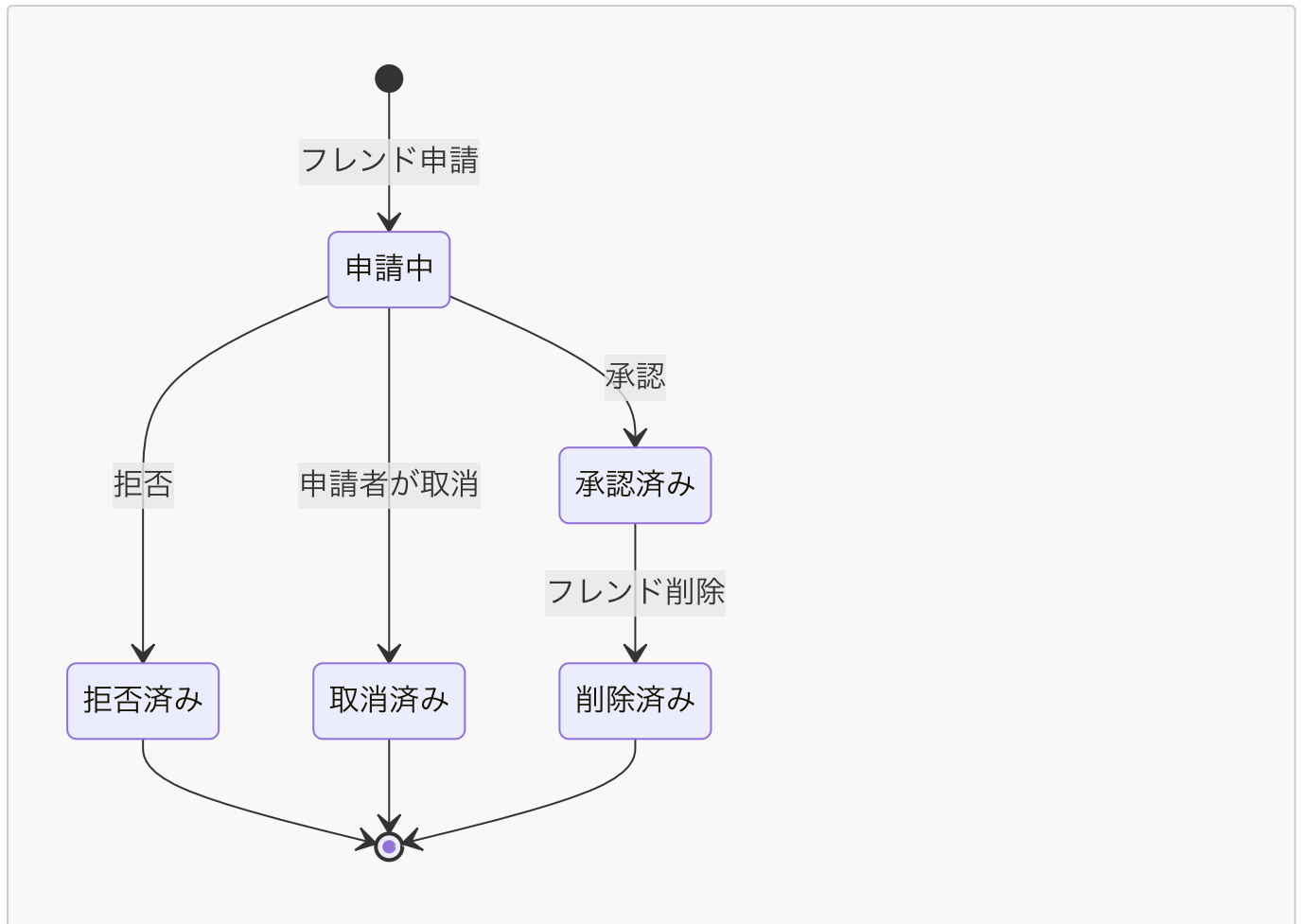
5. ドパの流れ



同じユーザーによる複数回送信は正常動作です。一方、通信再試行による意図しない二重加算と、ユーザーが意図して連打した複数回のドパをどう区別するかは、API設計時に決める必要があります。

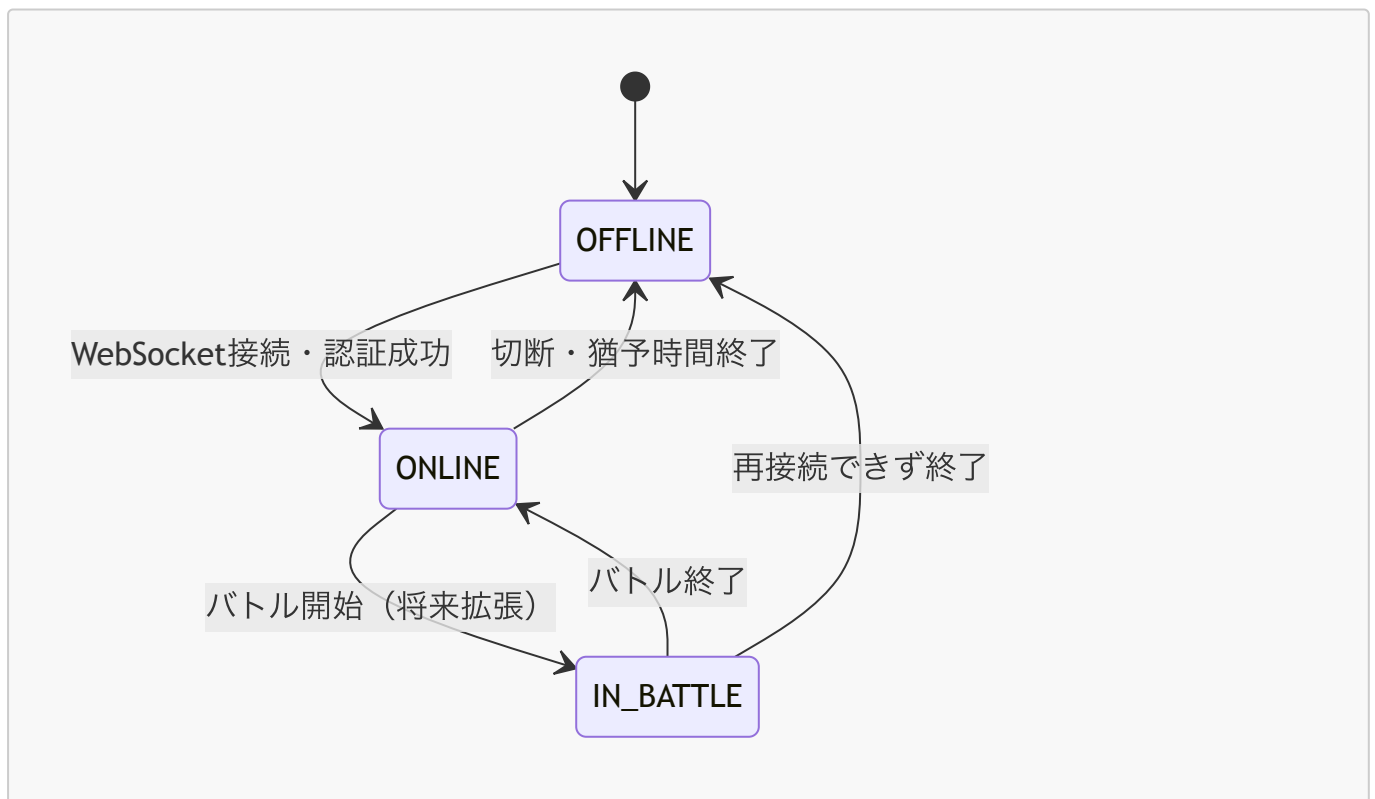
6. フレンドとオンライン状態

フレンド申請の状態



Friend.status の具体的な列挙値は原文で未確定です。実装時には、二重申請、相互申請、自分自身への申請をどう扱うかも固定します。

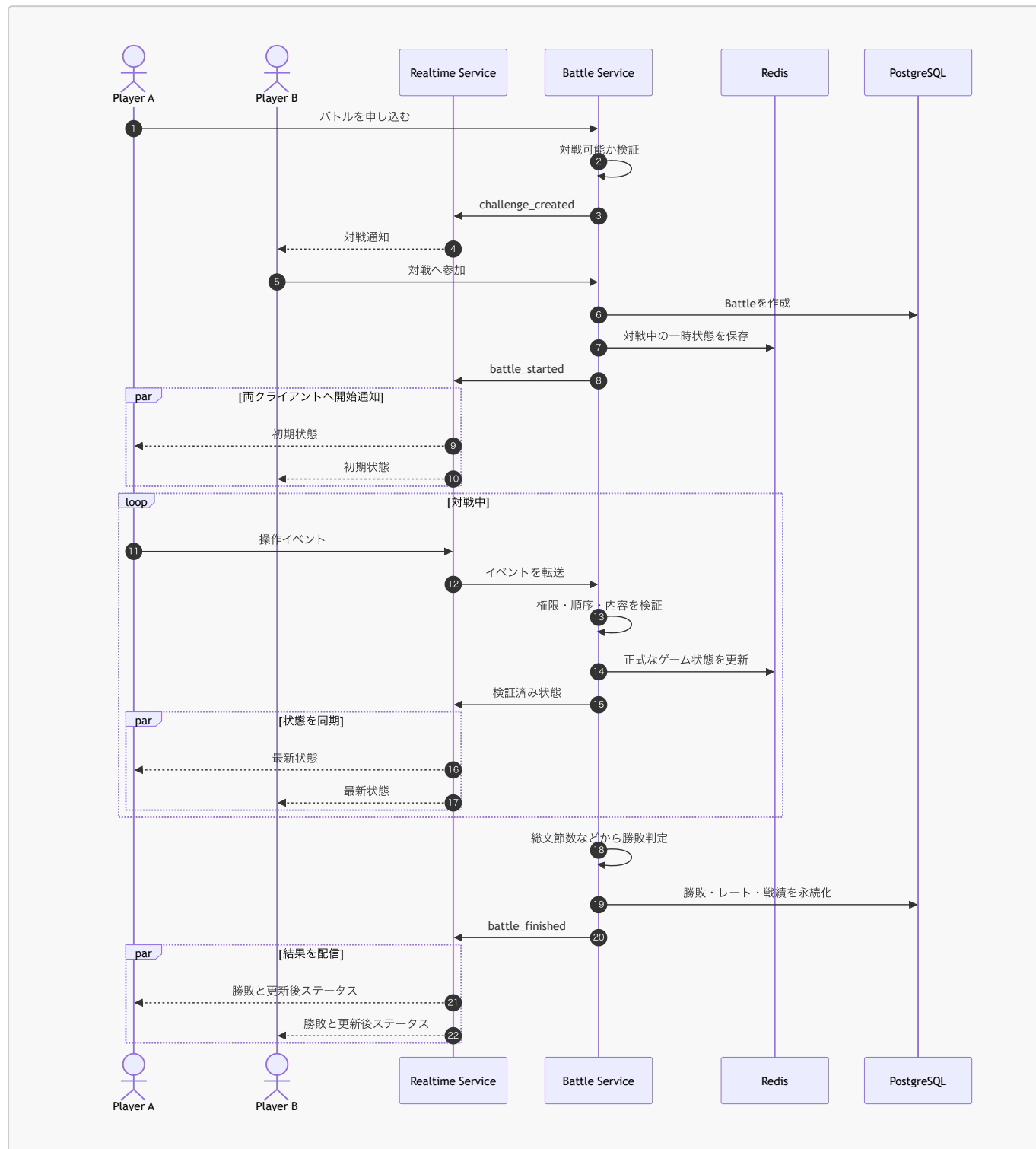
Presenceの状態



最低限必要なのは **ONLINE** と **OFFLINE** です。**IN_BATTLE** は原文上の将来案です。

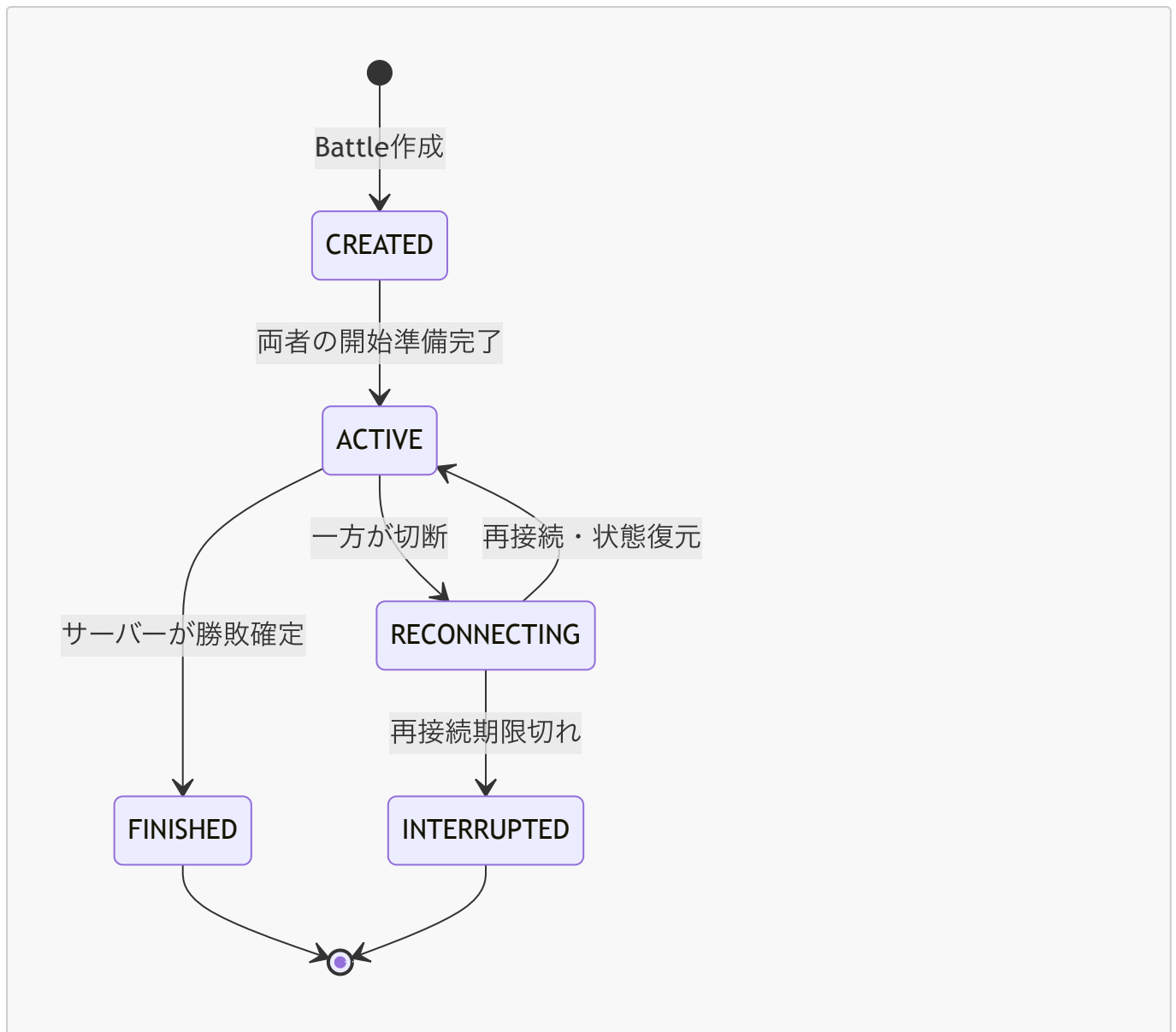
7. リアルタイムバトル

勝敗やゲーム状態はクライアントではなく、Battle Serviceを正とします。Realtime Serviceは接続と配信を担当し、勝敗ロジックの所有者にはなりません。



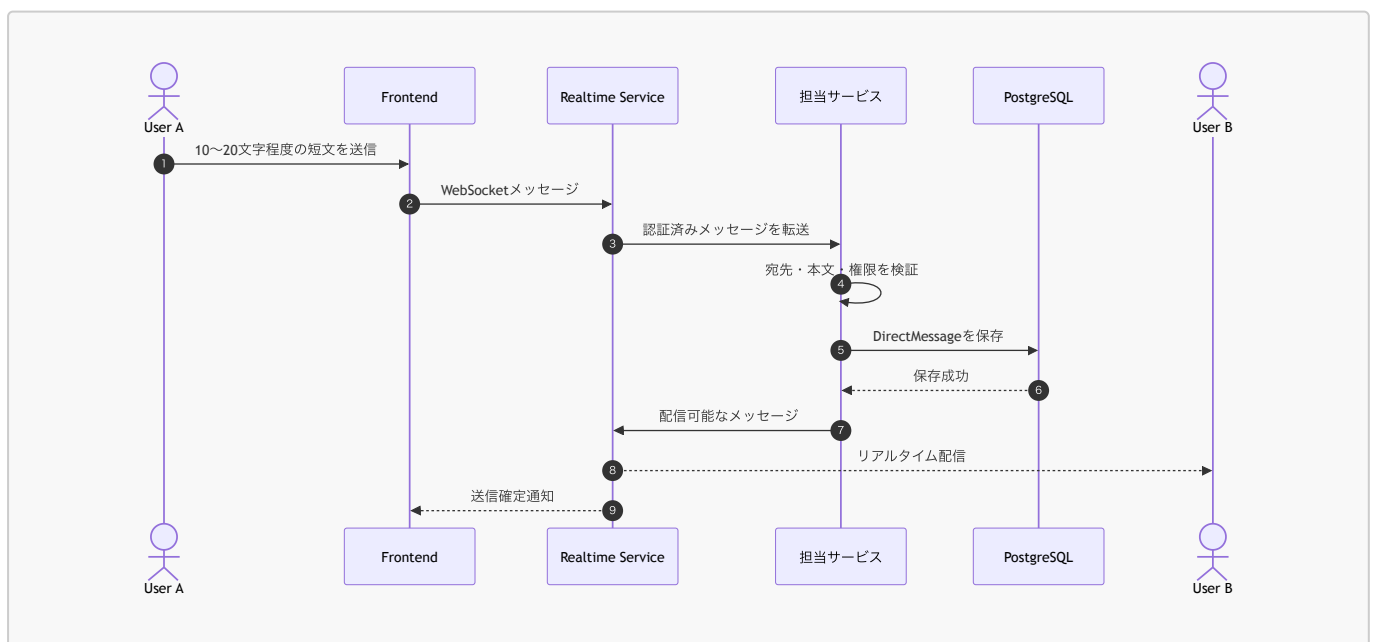
このシーケンスの参加・操作イベントの詳細は、原文の要件から組み立てた想定フローです。バトルの入力、制限時間、受諾・拒否、フォロワー奪取量は未定です。

切断と再接続



再接続猶予時間、期限切れ時の勝敗、イベントの連番・重複排除方式は、実装前に追加定義が必要です。

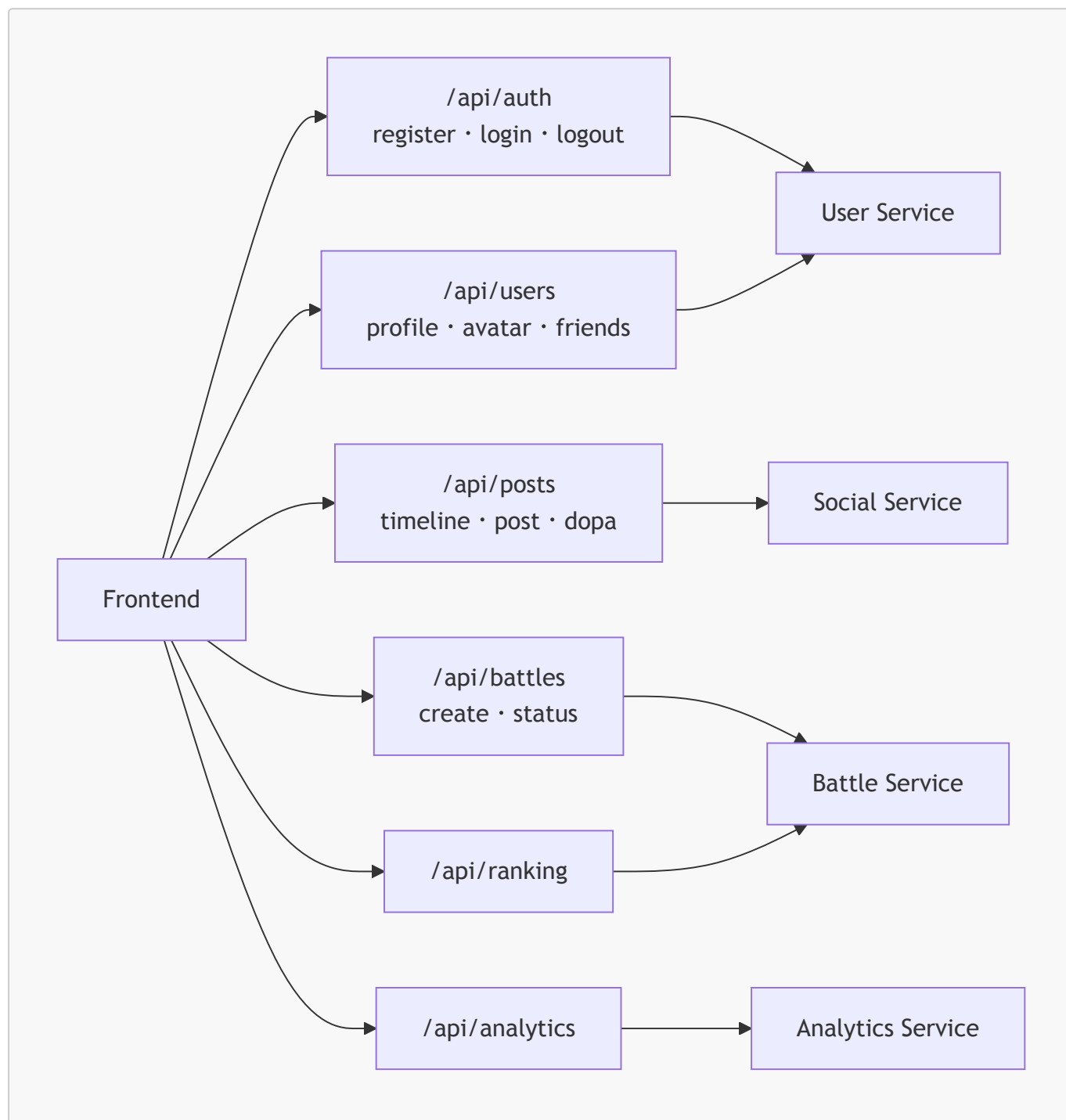
8. DM・チャット



原文ではDMをどのサービスが所有するかが明示されていないため、図中では「担当サービス」としてしています。実装前にUser、Social、専用Chat Serviceのいずれが所有するかを決めます。また「10～20文字程度」も厳密な最小・最大値へ確定する必要があります。

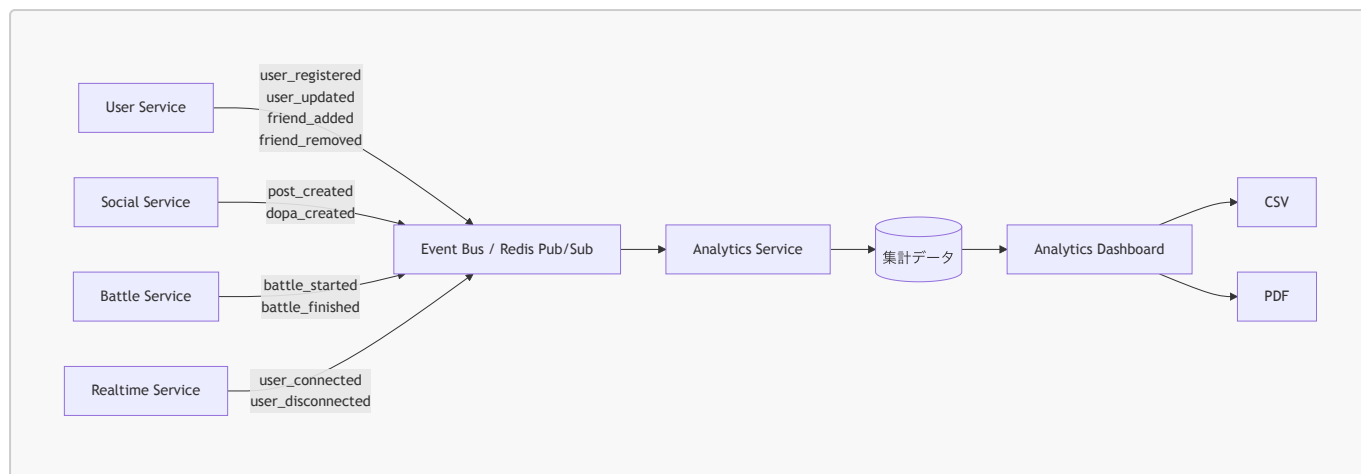
9. REST APIとイベント

REST APIの役割



原文のエンドポイントは設計例です。リクエスト・レスポンスSchema、エラー形式、ページネーション、認可条件は別途定義します。

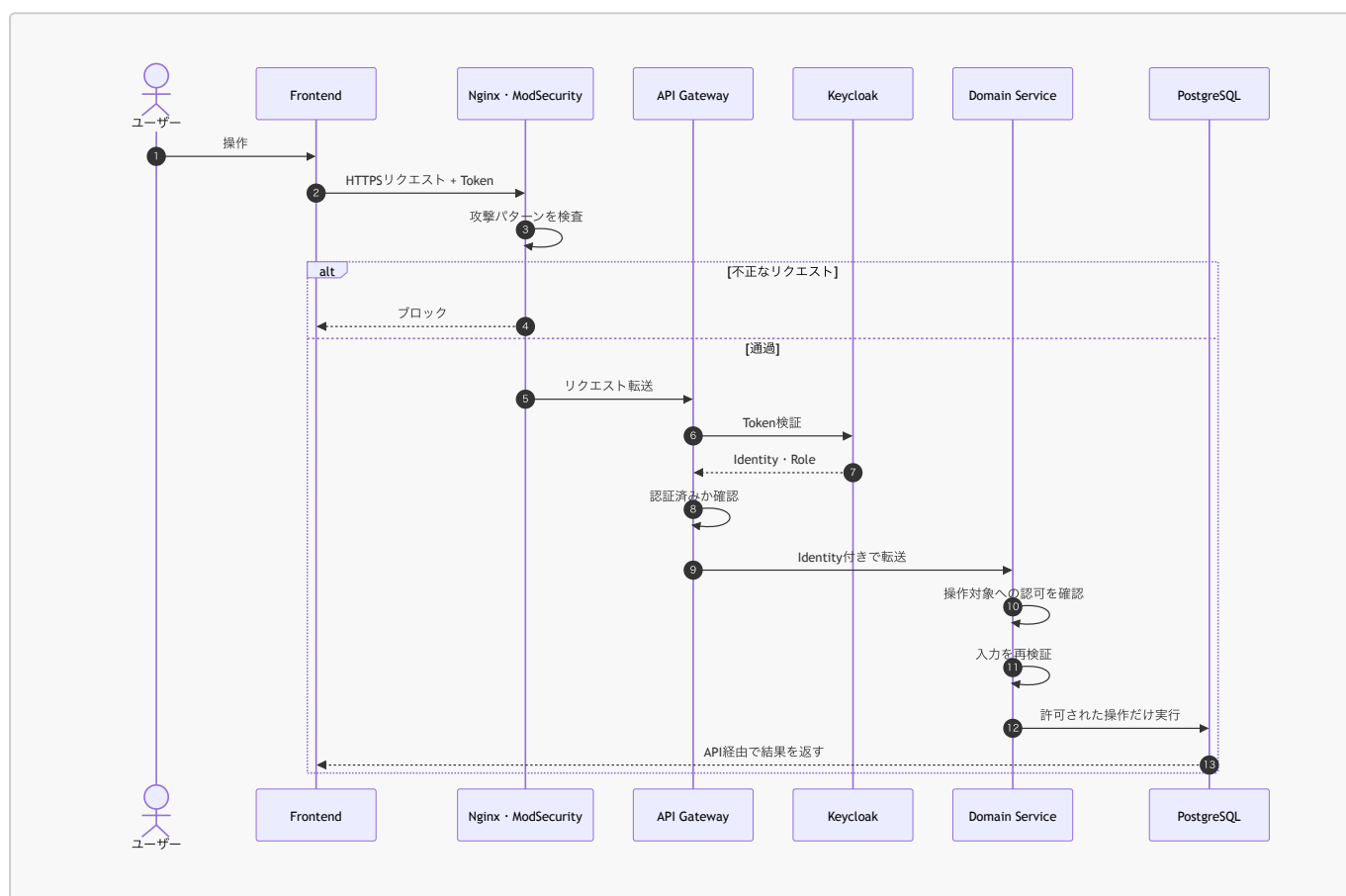
イベントが分析へ届く流れ



イベントにはID、発生日時、Schema versionを持たせ、Analytics側で同一イベントを二重集計しない設計が望めます。これは原文の「複数ユーザー」「二重送信」「整合性」の要件を満たすための実装上の注意です。

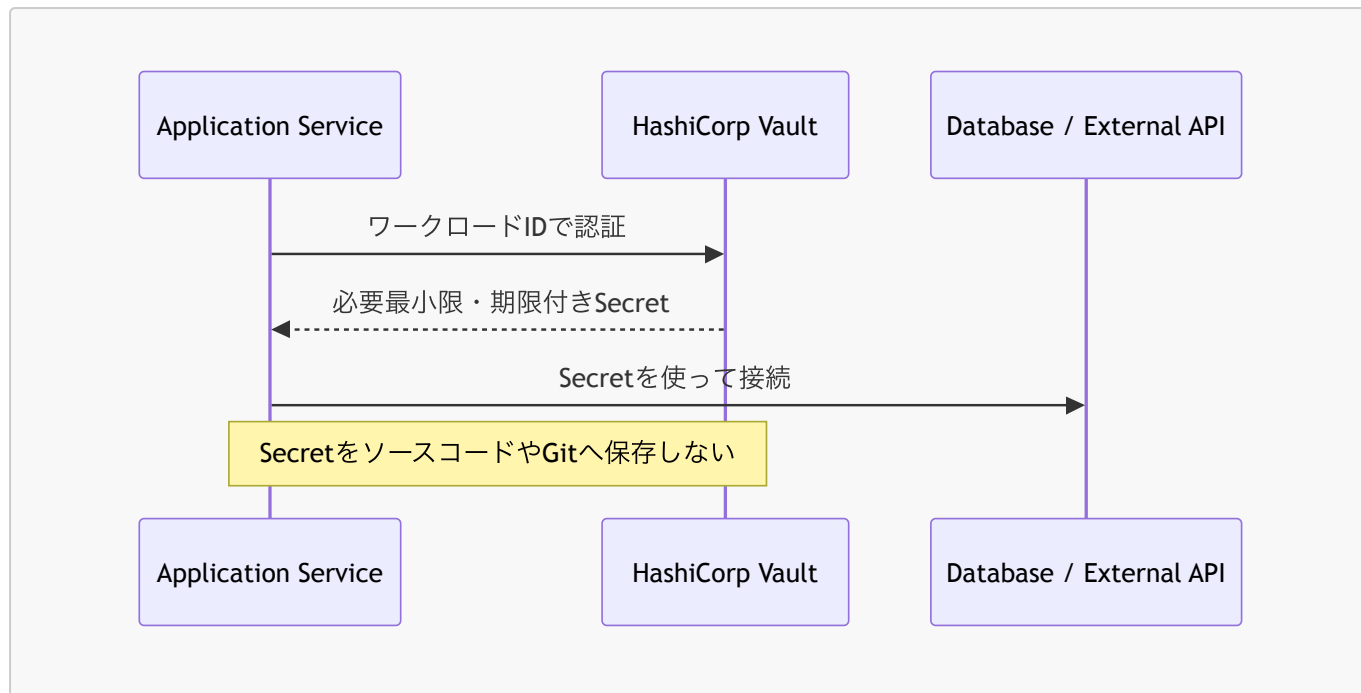
10. 認証・認可とセキュリティ

リクエストがデータへ到達するまで



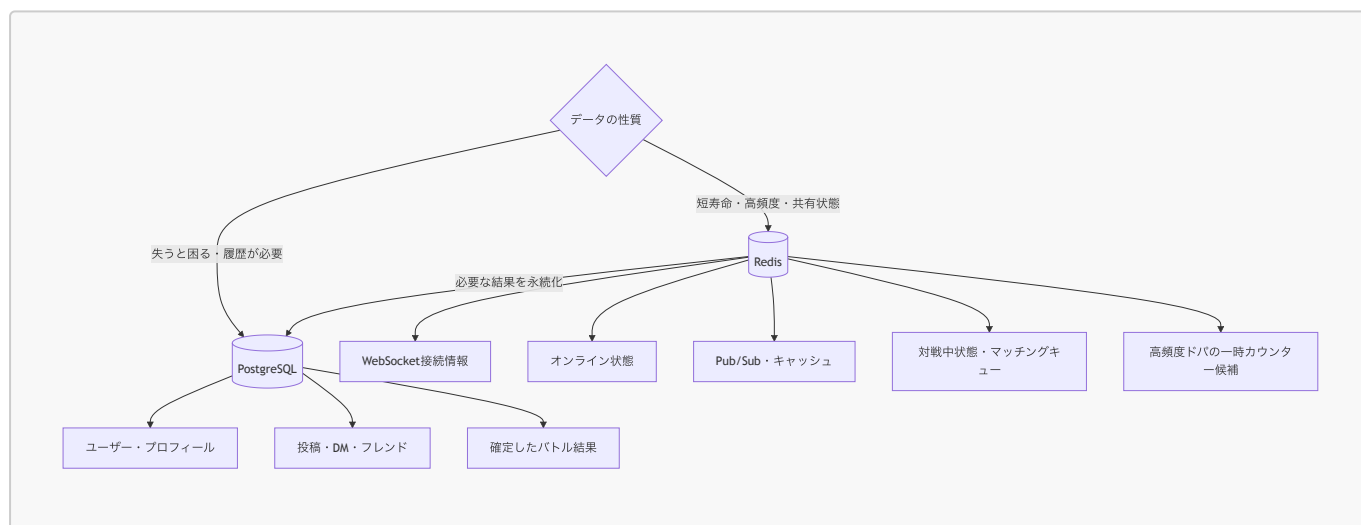
処理順は「認証 → 認可 → 入力検証 → リソースアクセス」です。他人のプロフィールを閲覧できることと、変更できることは別の権限として扱います。

Secretの供給

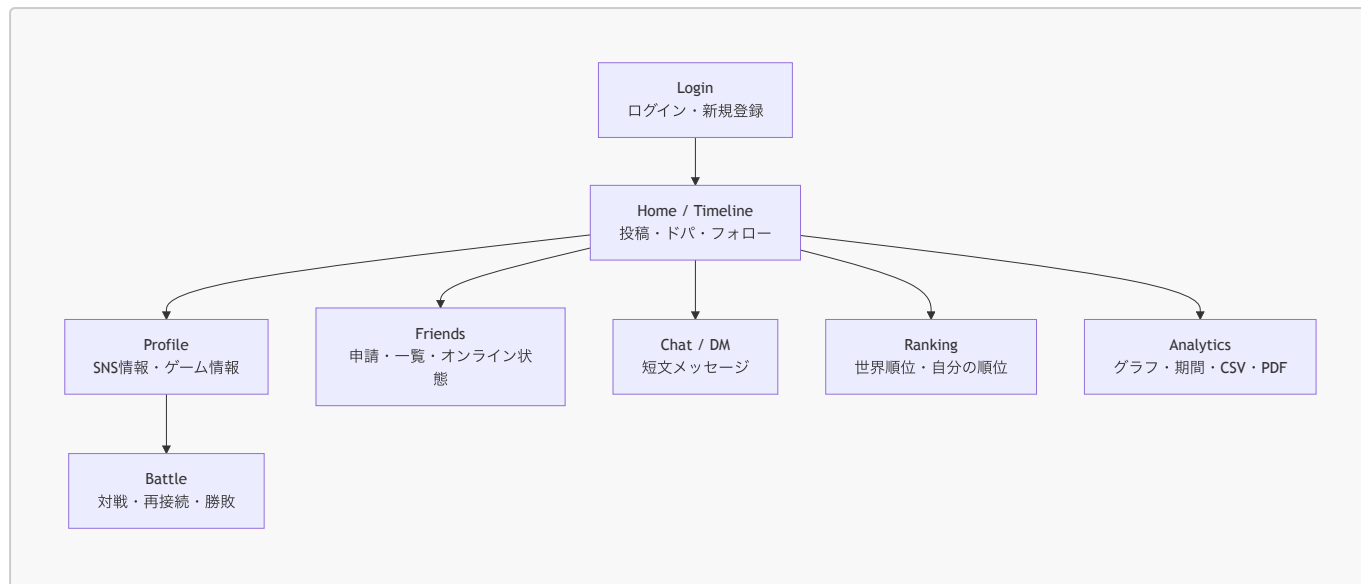


WAFはSQL Injection、XSS、Path Traversal、Command Injection、不正HTTPリクエストなどを防ぐ外側の防御です。WAFがあっても、アプリケーション側の認可、バリデーション、ORMの安全な利用は省略できません。

11. PostgreSQLとRedisの使い分け

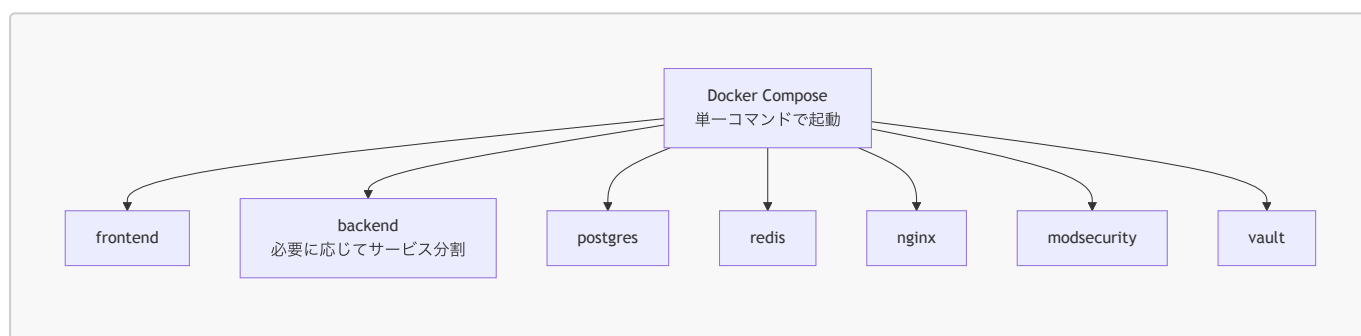


12. 画面と機能の対応



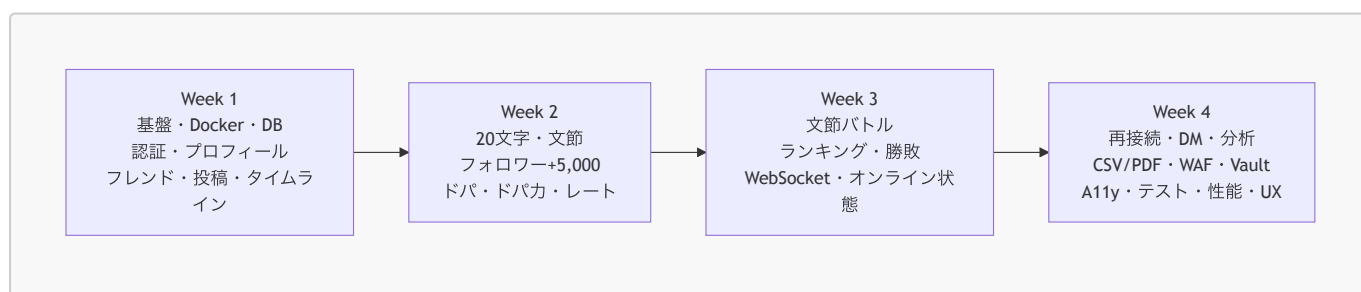
全画面で、キーボード操作、スクリーンリーダー、フォーカス、ラベル、エラー表示、コントラスト、アニメーションへの配慮が必要です。七色のRGBグローは演出に使えますが、状態や重要情報を色だけで伝えてはいけません。

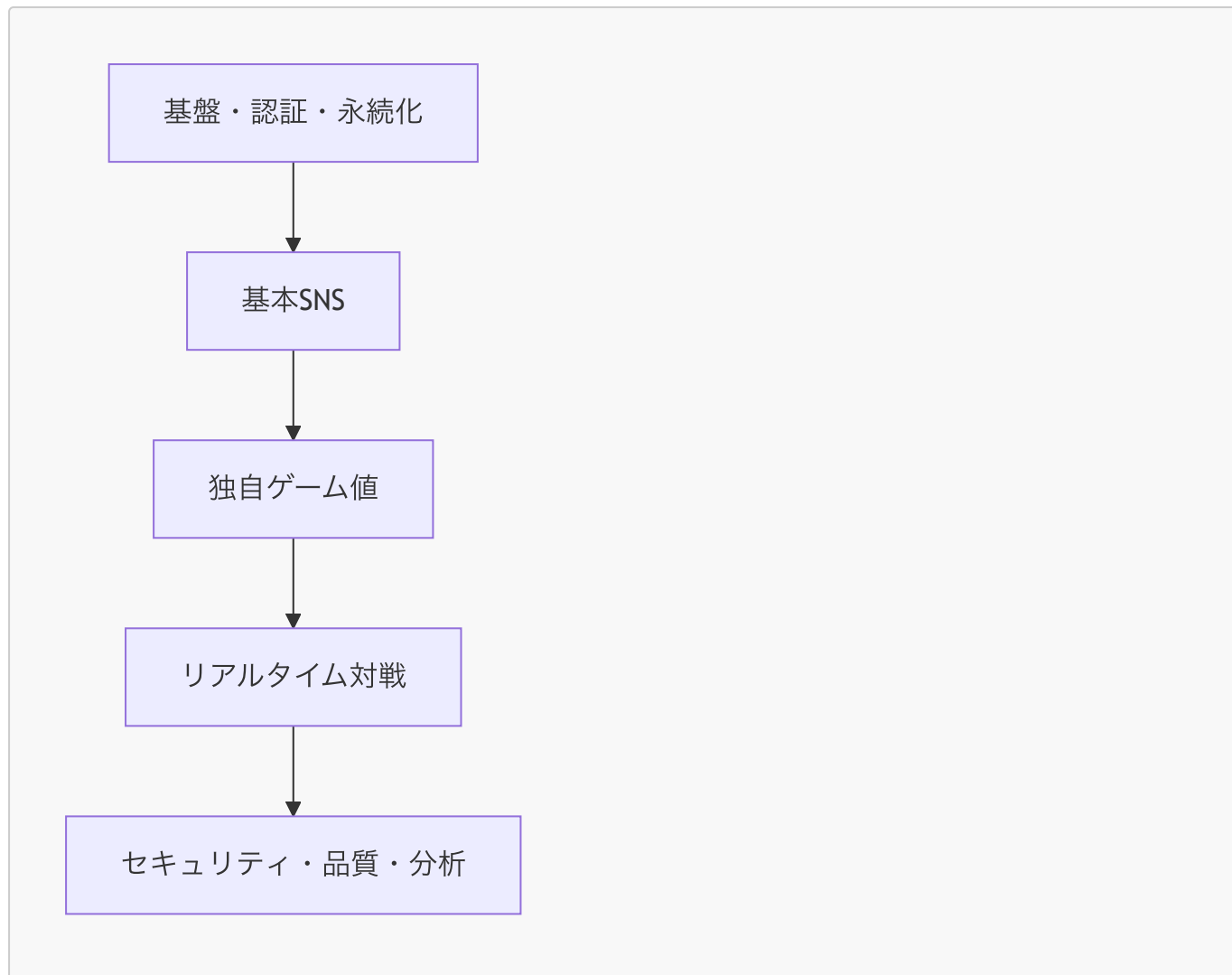
13. Dockerでの実行単位



フロントエンドはNext.js・React・TypeScript、バックエンドはNestJS・TypeScript、DBアクセスはPrismaを使用する想定です。

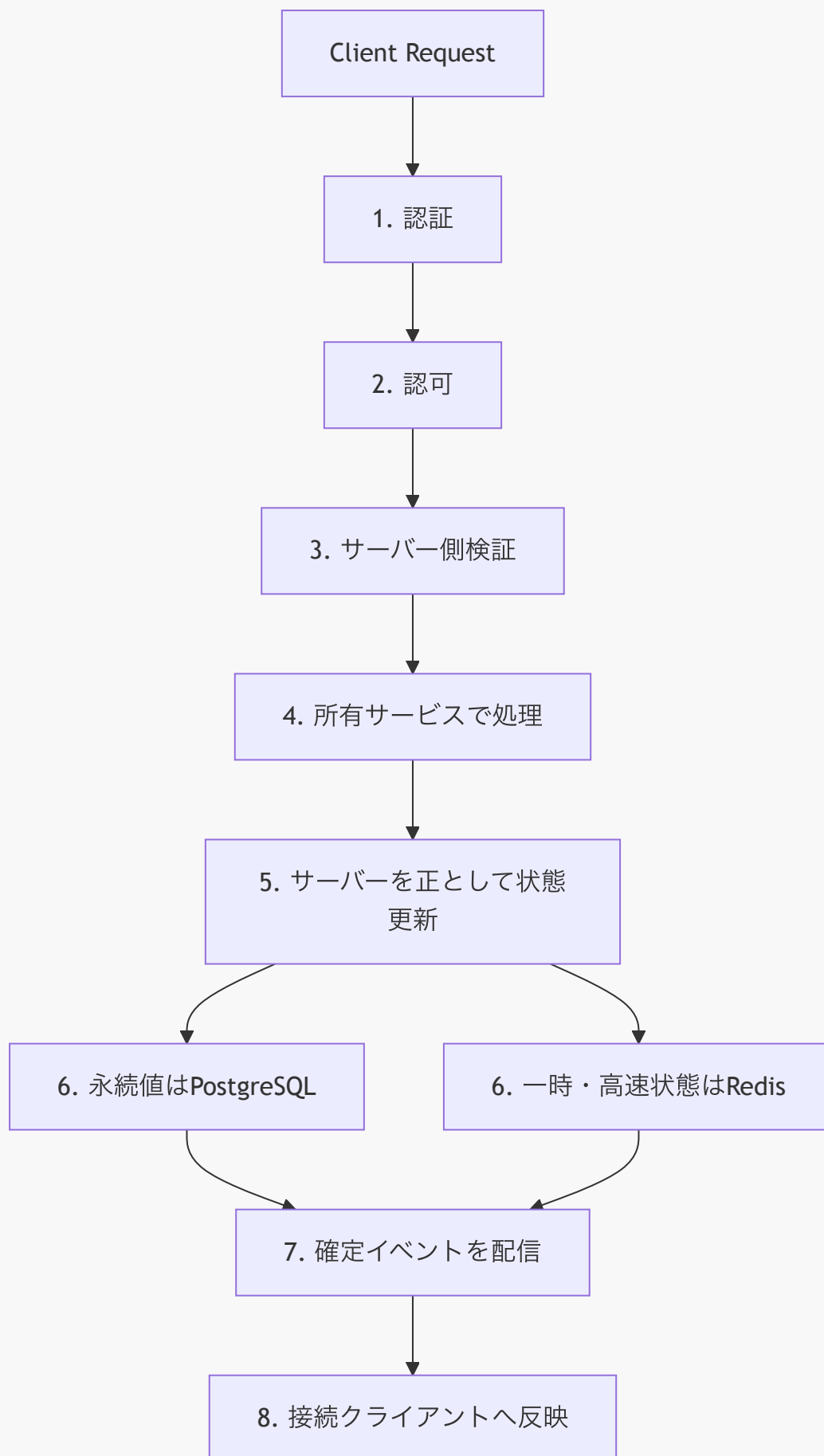
14. 4週間の開発順序





GitではIssueからブランチを作成し、Pull Requestでレビューしてから統合します。`main` への直接マージは禁止し、コミットメッセージには変更内容を明記します。

15. 実装時に守る原則



- ユーザーID、投稿文字数、ドバ数、バトル結果、レート、フォロワー数、権限をクライアント任せにしません。
- 重要なゲーム状態はServer Authoritativeにします。
- 各サービスの責務を混在させません。
- PostgreSQLとRedisの役割を分けます。
- 同時操作、再送、切断が起きてもデータ破損や二重処理が起きないようにします。

16. 原文で未確定の事項

実装に入る前に、少なくとも次を仕様として確定する必要があります。

項目	現在わかっていること	未確定部分
文節数	投稿時に算出し総文節数へ加算	日本語の分割方式、使用ライブラリ、絵文字・句読点の扱い
ドバカ	文節、レート、戦績、連勝数などで算出	正確な式、更新タイミング、上限
ドパレート	勝利で上昇、敗北で下降	初期値、増減式、引き分け・切断時の扱い
バトル	総文節数が多い側が勝利	受諾、制限時間、同値、フォロワー奪取量、不戦勝
再接続	切断・再接続へ対応	猶予時間、再送・順序保証、期限切れ時の結果
DM	10～20文字程度の短文	厳密な最小・最大値、所有サービス、既読・履歴取得
Friend.status	申請・承認・拒否・削除が必要	列挙値、相互申請、再申請、ブロックとの関係
Profileの所有	一つのProfileにSNS値とBattle値がある	サービス別テーブルへ分割するか、単一の更新担当を置くか
認証	Keycloak、OAuth 2.0・OIDC、User.passwordHashを想定	認証情報をKeycloakとUser Serviceのどちらが所有するか、Token保管・更新、サービス間認証
イベント	イベント名の一覧あり	Payload、version、順序、再試行、重複排除、保持期間
Analytics	リアルタイム集計と期間指定、CSV・PDF	指標定義、集計粒度、タイムゾーン、権限

17. 完成形

